

RAG

Thursday, February 12, 2026 12:28 AM

RAG - PRACTICAL STEPS

Project: Retrieval-Augmented Generation (RAG) System
Technology: Python, LangChain, ChromaDB, Ollama

PREREQUISITES

1. Python 3.10 or higher installed
2. Ollama installed from <https://ollama.ai/>
3. Internet connection for downloading models

STEP 1: INSTALL PYTHON DEPENDENCIES

Open terminal/command prompt in the project directory and run:

```
pip install -r requirements.txt
```

This will install the following libraries:

- pypdf : For reading PDF documents
- langchain : LLM framework for building RAG applications
- chromadb : Vector database for storing document embeddings
- pytest : For running test cases
- boto3 : AWS SDK (optional, for Bedrock embeddings)

STEP 2: INSTALL OLLAMA MODELS

Make sure Ollama is running, then pull the required models:

A) Embedding Model (for converting text to vectors):

```
ollama pull nomic-embed-text
```

B) LLM Model (for generating responses):

```
ollama pull mistral
```

STEP 3: ADD YOUR PDF DOCUMENTS

1. Navigate to the project folder
2. Open the "data" folder
3. Copy your PDF files into this folder
4. The system will process all PDFs in this directory

STEP 4: POPULATE THE DATABASE

Run the following command to process PDFs and create embeddings:

```
python populate_database.py
```

Expected output:

- Number of existing documents in DB: X
- Adding new documents: Y (if new PDFs found)

To RESET the database and reprocess all documents:

```
python populate_database.py --reset
```

STEP 5: QUERY THE RAG SYSTEM

Ask questions about your documents using:

```
python query_data.py "Your question here"
```

Example:

```
python query_data.py "who is Anshul at MITRC"
```

The system will:

1. Search for relevant context in your documents
2. Pass the context to the LLM
3. Generate and display the answer with sources

STEP 6: RUN TESTS (OPTIONAL)

To run the test cases:

```
pytest test_rag.py
```

PROJECT FILES OVERVIEW

File	Purpose
------	---------

get_embedding_function.py	Configures which embedding model to use
populate_database.py	Loads PDFs and stores them in ChromaDB

query_data.py	Handles user queries and generates responses
test_rag.py	Contains test cases for validation
requirements.txt	Lists all Python dependencies
data/	Folder for placing PDF documents
chroma/	Auto-generated vector database folder

=====

TROUBLESHOOTING

=====

Problem: OllamaEndpointNotFoundError

Solution: Run "ollama pull <model-name>" to download the missing model

Problem: Connection refused error

Solution: Make sure Ollama is running with "ollama serve"

Problem: No documents found

Solution: Verify PDF files are placed in the "data/" folder

Problem: AWS credentials error

Solution: The project uses Ollama by default; no AWS needed

=====

END OF DOCUMENT

=====